实验三 Mindspore 实现手写字体识别

姓名　顾凯琪

学号　226001624

学院 信息学院

专业班级 计算机 2 班

一、 实验目的

1. 加深对神经网络原理的理解
2. 熟悉 Minspore 平台
3. 掌握训练过程

二、 实验环境

1. Windows11
2. Anaconda 虚拟环境
3. 在上述 python 里安装好 mindspore 框架
4. (建议)在集成开发环境(IDE)—— PyCharm 上编写代码

三、 实验内容

1. 数据集加载

本次实验使用卷积神经网络（LeNet）进行多分类问题（手写数字识别）的求解，模型的训练与验证基于 MNIST 手写数字数据集进行，该数据集容易在互联网上获取，本次实验采用实验指导给出的二进制形式的数据集。

为了方便数据集的遍历与管理，实验代码实现了数据集类，其中类内部进行了数据集的加载、预处理、访问接口的代码编写，并实现了功能的封装，在调用处只需传入数据集路径、是否打乱、批次大小等信息即可完成数据集的加载，以供后续基于 MindSpore 框架的模型训练。

同时，为了方便对数据集信息进行查看，实验代码实现了可视化接口，该接口传入图像所对应的矩阵数组，输出对应的图像图片，以便于后续的模型测

试分析。

2. 模型的实现

对于神经网络模型，由于网络参数量与神经元数目巨大，手动实现网络结构与训练逻辑相对较为困难，为简化模型的实现过程，许多深度学习框架提供了神经网络的定义逻辑。在 MindSpore 中，继承自 nn.Cell 的类可以通过重写 construct 方法来定义网络的正向传播逻辑，在框架内部将自动根据正向传播逻辑的实现代码来计算梯度的反向传播路径。

对于手写识别任务,经典的卷积神经网络具有良好表现,本次实验使用 LeNet 进行手写数字识别。模型的逻辑代码采用 MindSpore 框架实现。

对于 LeNet，使用卷积层和池化层进行数据的特征提取，采用全连接层实现从特征到类别的关系映射，首先待识别图像将经过一个卷积层，后经过一层激活函数（ReLU）进行激活，再通过池化层减少特征尺度，重复以上操作 2 遍，得到高维的抽象特征，再将特征按行展开排列，使用 3 次全连接+激活的方式得到输出层，输出层的每一个神经元代表分类到各个类别的概率。

具体的是实现代码见报告第五节。

3. 检查点设置

为了保存模型的训练结果，需要进行检查点设置，对于 MindSpore，需要提前声明网络模型在训练过程中是否需要保存中间过程和结果，通过 CheckpointConfig 和 ModelCheckpoint 类进行检查点的信息配置,如模型的保存路径、检查点保存时刻、文件命名等信息。

4. 模型训练参数的配置

模型的训练需要确定一系列参数，包括训练轮次，学习率，数据批量大小，损失函数等。

对于损失函数，多分类问题中常用交叉熵损失函数，其表达式如下：

$$Loss = -\sum_{i} y_i \log(f(x_i))$$

其中 yi 表示真实情况下是否为该类，即只取预测为对应类别的概率，f 为模型的输出概率。

学习率方面，初始不宜设置太大，本次实验选用 0.01。对于训练过程，深度

学习框架通常提供对应的优化器以加速训练过程，例如 MindSpore 中的 Momentum 优化器，需要传入模型参数（使用 trainable_params()方法访问）以及学习率。

以上所有的相关训练参数最终通过 mindspore.Model 类进行包装，返回一个可训练的模型对象，调用该对象的 train 函数并传入数据集、epochs、检查点信息即可进行训练。

5. 模型训练与验证

调用 train 函数即可实现模型的训练，同步会将训练过程的相关信息进行打印，使用 eval 函数测试模型在测试集上的表现。完成训练后能够得到 checkpoint，使用 load_checkpoint 可以载入对应检查点中的模型参数，使用 load_param_into_net 可以将参数加载到指定的网络实例中，测试一个批次的数据，进行可视化并查看模型的输出情况。

6. 模型参数调整

对模型的参数进行调整，以得到更优的测试集准确率。

四、 实验步骤：

1. 下载压缩包里的数据集以及数据集加载文件夹放置目录如下：



2. 安装 Anaconda 软件并使用 conda 命令创建 python3.xx 环境；

```
C:\Users\asus>conda activate MindSpore

(MindSpore) C:\Users\asus>python --version
Python 3.11.11

(MindSpore) C:\Users\asus>pip install https://ms-release.obs.cn-north-4.myhuaweicloud.com/2.4.10/MindSpore/cpu/x86_64/mindspore-2.4.10-cp311-cp311-win_amd64.whl --trusted-host ms-release.obs.cn-north-4.myhuaweicloud.com -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting mindspore==2.4.10
  Downloading https://ms-release.obs.cn-north-4.myhuaweicloud.com/2.4.10/MindSpore/cpu/x86_64/mindspore-2.4.10-cp311-cp311-win_amd64.whl (100.4 MB)
     ---------------------------------------- 100.4/100.4 MB 10.6 MB/s eta 0:00:00
Collecting numpy<2.0.0,>=1.20.0 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/3f/6b/5610004206cf7f8e7ad91c5a85a8c71b2f2f8051a0c0c4d5916b76d6cbb2/numpy-1.26.4-cp311-cp311-win_amd64.whl (15.8 MB)
     ---------------------------------------- 15.8/15.8 MB 11.3 MB/s eta 0:00:00
Collecting protobuf>=3.13.0 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/61/fa/aae8e10512b83de633f2646506a6d835b151edf4b30d18d73afd01447253/protobuf-5.29.3-cp310-abi3-win_amd64.whl (434 kB)
Collecting asttokens>=2.0.4 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/25/8a/c46dcc25341b5bce5472c718902eb3d38600a903b14fa6aeecef3f21a46f/asttokens-3.0.0-py3-none-any.whl (26 kB)
Collecting pillow>=6.2.0 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/3a/c6/fce9255272bcf0c39e15abd2f8fd8429a954cf344469eaceb9d0d1366913/pillow-11.1.0-cp311-cp311-win_amd64.whl (2.6 MB)
     ---------------------------------------- 2.6/2.6 MB 10.1 MB/s eta 0:00:00
Collecting scipy>=1.5.4 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/b9/8b/7ec1832b09dbc88f3db411f8cdd47db04505c4b72c99b11c920a8f0479c3/scipy-1.15.2-cp311-cp311-win_amd64.whl (41.2 MB)
     ---------------------------------------- 41.2/41.2 MB 11.2 MB/s eta 0:00:00
Collecting packaging>=20.0 (from mindspore==2.4.10)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/88/ef/eb23f262cca3c0c4eb7ab1933c3b1f03d021f2c48f54763065b6f0e321be/packaging-24.2-py3-none-any.whl (65 kB)
Collecting psutil>=5.6.1 (from mindspore==2.4.10)
```

3. 在上述环境中安装好相关的依赖库，特别是 mindspore 库；

4. 参照代码 pdf 文件和 ppt 里的步骤编写 mnist 手写数据集代码，完成训练与预测，在 word 中粘贴可视化结果图；

5. 尝试通过调整模型结构 or 参数 or 超参数 or 权重初始化方式（训练 epoch 最大不能超过 30）使得测试集准确率超过 Baseline，最好准确率需要≥97%。粘贴测试准确率截图，如下图：

```
Cmder
Epoch:[ 43/ 50], step:[ 1875/ 1875], loss:[0.000/0.001], time:25.930 ms, lr:0.01000
Epoch time: 21087.670 ms, per step time: 11.247 ms, avg loss: 0.001
Epoch:[ 44/ 50], step:[ 1875/ 1875], loss:[0.000/0.003], time:15.953 ms, lr:0.01000
Epoch time: 21447.649 ms, per step time: 11.439 ms, avg loss: 0.003
Epoch:[ 45/ 50], step:[ 1875/ 1875], loss:[0.000/0.005], time:16.954 ms, lr:0.01000
Epoch time: 19525.787 ms, per step time: 10.414 ms, avg loss: 0.005
Epoch:[ 46/ 50], step:[ 1875/ 1875], loss:[0.000/0.001], time:20.944 ms, lr:0.01000
Epoch time: 20107.233 ms, per step time: 10.724 ms, avg loss: 0.001
Epoch:[ 47/ 50], step:[ 1875/ 1875], loss:[0.000/0.001], time:19.947 ms, lr:0.01000
Epoch time: 26344.553 ms, per step time: 14.050 ms, avg loss: 0.001
Epoch:[ 48/ 50], step:[ 1875/ 1875], loss:[0.000/0.002], time:17.953 ms, lr:0.01000
Epoch time: 19010.166 ms, per step time: 10.139 ms, avg loss: 0.002
Epoch:[ 49/ 50], step:[ 1875/ 1875], loss:[0.000/0.004], time:15.957 ms, lr:0.01000
Epoch time: 19622.528 ms, per step time: 10.465 ms, avg loss: 0.004
[WARNING] ME(5956:11396,MainProcess):2022-03-30-23:35:46.536.84 [mindspore\train\mody:954] CPU cannot support dataset sink mode currently.So the evaluating process will performed with dataset non-sink mode.
{'accuracy': 0.9910857371794872}
```

6. 用自己的话解释代码中主要函数的意义。（输入、输出以及做了什么工作，不必非常精确，体现自己的理解为主、不要单纯复制粘贴已有信息

或网上资料）

五、 代码提交

```
"""
-------相关库导入-----
"""
```

```
# 相关库导入
from mindvision.dataset import Mnist
import mindspore.nn as nn
import mindspore as ms
from mindvision.engine.callback import LossMonitor
import numpy as npimport matplotlib.pyplot as plt
```

```
"""
-------数据集的导入-----
"""
```

```
# 数据集准备
download_train = Mnist(
    path="./mnist",
    split="train",
    batch_size=32,
    repeat_num=1,
    shuffle=True,
    resize=32,
    download=True)
download_eval = Mnist(
    path="./mnist",
    split="test",
    batch_size=32,
    resize=32,
    download=True)
dataset_train = download_train.run()
dataset_eval = download_eval.run()
```

```
"""
------- LeNet 的实现-----
"""
```

```
# 随机种子
seed_value = 2025
model = RandomForestClassifier(random_state=seed_value)
```

```
# 模型训练
model.fit(x_train, y_train)
# 在训练集、测试集上进行预测
y_train_pred = model.predict(x_train)
y_test_pred = model.predict(x_test)
# 模型定义
class LeNet5(nn.Cell):
    def __init__(self, num_class=10, num_channel=1):
        super(LeNet5, self).__init__()
        # 卷积层，输入通道数为num_channel，输出通道数为6，卷积核大小5*5
        self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
        # 卷积层，输入的通道数为6，输出的通道数为16，卷积核大小为5*5
        self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
        # 全连接层，输入个数为16*5*5，输出个数为120
        self.fc1 = nn.Dense(16 * 5 * 5, 120)
        # 全连接层，输入个数为120，输出个数为84
        self.fc2 = nn.Dense(120, 84)
        # 全连接层，输入个数为84，分类的个数为num_class
        self.fc3 = nn.Dense(84, num_class)
        # ReLU 激活函数
        self.relu = nn.ReLU()
        # 池化层
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        # 多维数组展平为一维数组
        self.flatten = nn.Flatten()

    def construct(self, x):
        # 使用定义好的运算构建前向网络
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x
```

```
"""
-------训练参数配置与模型训练-----
"""
```

```
# 模型实例化
net=LeNet5(num_class=10)
# 损失函数：交叉熵损失
loss=nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
# 优化器
optimizer=nn.Momentum(net.trainable_params(),
                      learning_rate=0.01, momentum=0.9)
# MindSpore 需要提前声明网络模型在训练过程中是否需要保存中间过程和结果
# 设置模型保存参数
config_ck = ms.CheckpointConfig(save_checkpoint_steps=1875, keep_checkp
oint_max=10)
# 应用模型保存参数
checkpoint = ms.ModelCheckpoint(prefix="lenet", directory="./lenet", co
nfig=config_ck)
# 模型训练初始化
model = ms.Model(network=net,
                 loss_fn=loss,
                 optimizer=optimizer,
                 metrics={'accuracy'})
# 训练网络模型
model.train(10, dataset_train,
                callbacks=[checkpoint, LossMonitor(0.01, 1875)])
```

```
"""
-------模型测试-----
"""
```

```
# 在测试集上进行模型的表现测试
acc = model.eval(dataset_eval)
print("{}".format(acc))
# 加载模型的参数
param_dict = ms.load_checkpoint("./lenet/lenet-1_1875.ckpt")
# 加载参数到网络中
ms.load_param_into_net(net, param_dict)
# 测试模型并可视化结果
mnist = Mnist("./mnist", split="train", batch_size=9, resize=32)
dataset_infer = mnist.run()
ds_test = dataset_infer.create_dict_iterator()
data = next(ds_test)
```

```
images = data["image"].asnumpy()
labels = data["label"].asnumpy()
plt.figure()
for i in range(1, 10):
    plt.subplot(3, 3, i)
    plt.imshow(images[i-1][0], interpolation="None", cmap="gray")
plt.show()
# 使用函数 model.predict 预测 image 对应分类
output = model.predict(ms.Tensor(data['image']))
predicted = np.argmax(output.asnumpy(), axis=1)
# 输出预测分类与实际分类
print(f'Predicted: "{predicted}", Actual: "{labels}"')
```

六、 测试集可视化和准确率实验结果分析

该部分将对实验代码产出的相关结果与图像进行记录与分析。

1. 数据可视化

给定的数据为 28*28 的图像，使用 matplotlib 库进行绘图，选取 9 个样本数据，绘图结果如图 6.1 所示。可以看到数据集以 28*28 的图像形式展现手写数字，也是神经网络的输入。

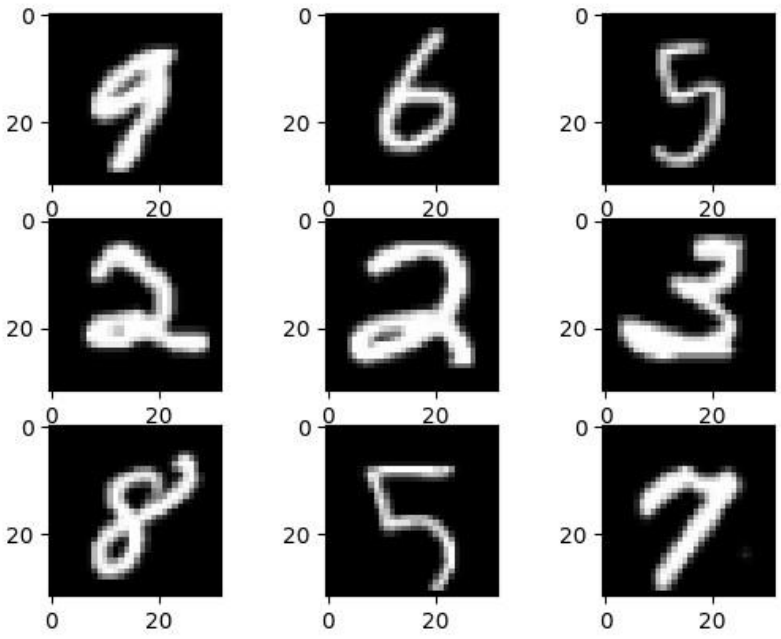


图 6.1 MNIST 手写数字数据可视化

2. 数据集的导入

运行代码，实现数据集的导入，使用 len 查看数据集的规模，如图 6.2 所示。

其中训练集包含 60000 个样本，测试集包含 10000 个样本。

```
D:\Anaconda\envs\MindSpore\python.exe "C:\Users\asus\Desktop\
train_len: 60000
eval_len: 10000
```

图 6.2 数据集大小

4. 模型训练过程

运行框架的模型训练函数，模型开始训练，并打印了训练过程，如图 6.3 所示。可以看到每一个 epoch 遍历训练完一遍训练集，损失逐渐下降，但由于学习率相对较高，导致在训练到第 17 轮的时候，损失值已经变成 nan，代表产生了梯度爆炸，因此此时需要将学习率相对降低，使得模型学习更多的细节。修改完的模型经历 50 轮训练，loss 从 2.301 降低至 0.05

```
Epoch:[ 12/ 50], step:[ 1875/ 1875], loss:[0.000/0.010], time:15.749 ms, lr:0.01000
Epoch time: 14563.962 ms, per step time: 7.767 ms, avg loss: 0.010
Epoch:[ 13/ 50], step:[ 1875/ 1875], loss:[0.000/0.010], time:15.317 ms, lr:0.01000
Epoch time: 14666.838 ms, per step time: 7.822 ms, avg loss: 0.010
Epoch:[ 14/ 50], step:[ 1875/ 1875], loss:[0.013/0.007], time:18.409 ms, lr:0.01000
Epoch time: 15412.610 ms, per step time: 8.220 ms, avg loss: 0.007
Epoch:[ 15/ 50], step:[ 1875/ 1875], loss:[0.000/0.008], time:17.287 ms, lr:0.01000
Epoch time: 16275.184 ms, per step time: 8.680 ms, avg loss: 0.008
Epoch:[ 16/ 50], step:[ 1875/ 1875], loss:[  nan/  nan], time:17.201 ms, lr:0.01000
Epoch time: 16137.647 ms, per step time: 8.607 ms, avg loss:   nan
Epoch:[ 17/ 50], step:[ 1875/ 1875], loss:[  nan/  nan], time:21.340 ms, lr:0.01000
Epoch time: 15393.040 ms, per step time: 8.210 ms, avg loss:   nan
Epoch:[ 18/ 50], step:[ 1875/ 1875], loss:[  nan/  nan], time:17.245 ms, lr:0.01000
Epoch time: 15592.402 ms, per step time: 8.316 ms, avg loss:   nan
Epoch:[ 19/ 50], step:[ 1875/ 1875], loss:[  nan/  nan], time:21.255 ms, lr:0.01000
Epoch time: 16380.744 ms, per step time: 8.736 ms, avg loss:   nan
```

```
Epoch:[  0/ 30], step:[ 1875/ 1875], loss:[2.297/2.301], time:15.358 ms, lr:0.00100
Epoch time: 16068.843 ms, per step time: 8.570 ms, avg loss: 2.301
Epoch:[  1/ 30], step:[ 1875/ 1875], loss:[2.307/2.299], time:16.157 ms, lr:0.00100
Epoch time: 16316.395 ms, per step time: 8.702 ms, avg loss: 2.299
Epoch:[  2/ 30], step:[ 1875/ 1875], loss:[2.296/2.296], time:19.879 ms, lr:0.00100
Epoch time: 15997.357 ms, per step time: 8.532 ms, avg loss: 2.296
Epoch:[  3/ 30], step:[ 1875/ 1875], loss:[2.280/2.289], time:18.296 ms, lr:0.00100
Epoch time: 16056.231 ms, per step time: 8.563 ms, avg loss: 2.289
Epoch:[  4/ 30], step:[ 1875/ 1875], loss:[2.207/2.262], time:20.181 ms, lr:0.00100
Epoch time: 15888.927 ms, per step time: 8.474 ms, avg loss: 2.262
Epoch:[  5/ 30], step:[ 1875/ 1875], loss:[1.188/1.826], time:18.297 ms, lr:0.00100
Epoch time: 28626.519 ms, per step time: 15.267 ms, avg loss: 1.826
Epoch:[  6/ 30], step:[ 1875/ 1875], loss:[0.471/0.810], time:13.730 ms, lr:0.00100
Epoch time: 16920.035 ms, per step time: 9.024 ms, avg loss: 0.810
Epoch:[  7/ 30], step:[ 1875/ 1875], loss:[0.510/0.520], time:14.271 ms, lr:0.00100
Epoch time: 16221.788 ms, per step time: 8.652 ms, avg loss: 0.520
Epoch:[  8/ 30], step:[ 1875/ 1875], loss:[0.310/0.413], time:14.300 ms, lr:0.00100
Epoch time: 15974.602 ms, per step time: 8.520 ms, avg loss: 0.413
Epoch:[  9/ 30], step:[ 1875/ 1875], loss:[0.569/0.350], time:13.136 ms, lr:0.00100
Epoch time: 16172.928 ms, per step time: 8.626 ms, avg loss: 0.350
Epoch:[ 10/ 30], step:[ 1875/ 1875], loss:[0.101/0.304], time:16.135 ms, lr:0.00100
Epoch time: 16120.496 ms, per step time: 8.598 ms, avg loss: 0.304
Epoch:[ 11/ 30], step:[ 1875/ 1875], loss:[0.158/0.267], time:15.314 ms, lr:0.00100
Epoch time: 16302.135 ms, per step time: 8.694 ms, avg loss: 0.267
Epoch:[ 12/ 30], step:[ 1875/ 1875], loss:[0.296/0.236], time:16.407 ms, lr:0.00100
Epoch time: 16865.605 ms, per step time: 8.995 ms, avg loss: 0.236
Epoch:[ 13/ 30], step:[ 1875/ 1875], loss:[0.237/0.210], time:14.736 ms, lr:0.00100
Epoch time: 16580.839 ms, per step time: 8.843 ms, avg loss: 0.210
Epoch:[ 14/ 30], step:[ 1875/ 1875], loss:[0.048/0.190], time:20.398 ms, lr:0.00100
Epoch time: 16829.276 ms, per step time: 8.976 ms, avg loss: 0.190
Epoch:[ 15/ 30], step:[ 1875/ 1875], loss:[0.156/0.173], time:13.840 ms, lr:0.00100
Epoch time: 16095.145 ms, per step time: 8.584 ms, avg loss: 0.173
Epoch:[ 16/ 30], step:[ 1875/ 1875], loss:[0.144/0.159], time:15.134 ms, lr:0.00100
```

图 6.3 模型训练过程

4. 模型测试

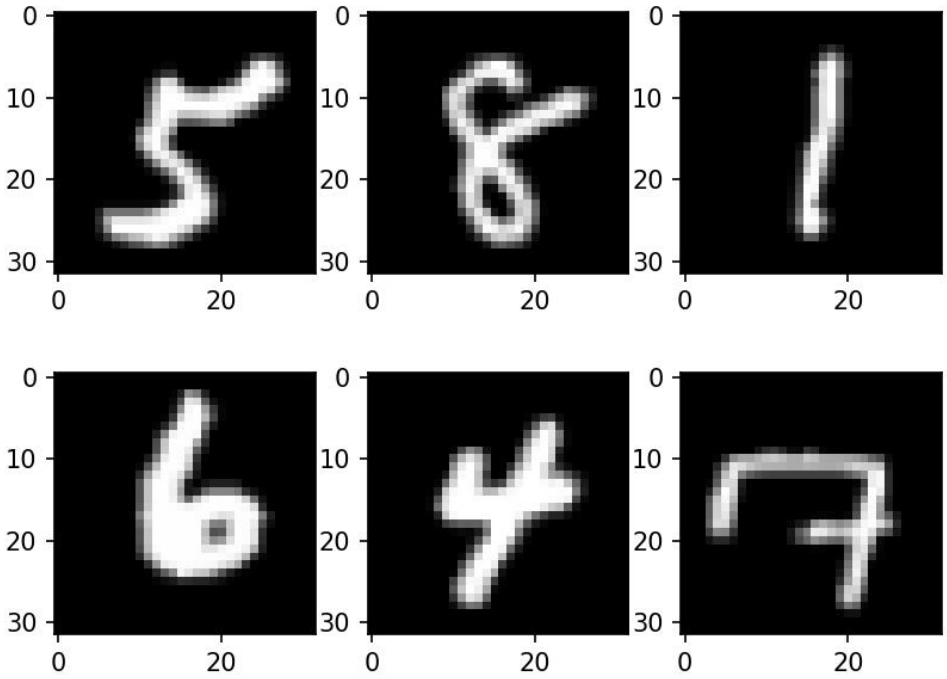
使用 eval 函数进行模型测试，使用测试集进行模型的推理，该函数直接返回最终的准确率，结果如图 6.4 所示。可以看到模型的准确率达到了 98.32%，已有较好的模型表现。

```
{'accuracy': 0.9831730769230769}
```

图 6.4 模型测试

由于前面训练过程中已保存了一系列 checkpoints，可以直接加载模型进行进一步的测试。加载模型后，随机选定测试集中的 6 个样本，显示对应的图片、标签，并输出模型的预测结果，如图 6.5 所示，尽管模型的准确率已经

在 98.32%以上，但最后一个相对扭曲的数字仍然错误的进行了判断。



```
Predicted: "[5 8 1 6 4 9]", Actual: "[5 8 1 6 4 7]"
```

图 6.5 部分样本的测试

可以看到，模型的预测结果与真实情况吻合，能够较好的对手写数字图片进行分类，具有良好性能。

通过调整超参数，将训练批次增加，并适当减小学习率以求达到更精确的最优解，结果如图 6.6 所示。准确率相对图 6.6 略有提升，达到 98.75%的准确率，但提升不明显，事实上模型的精确度已经较高了，继续训练可能会降低模型的泛化性能。

```
{'accuracy': 0.9875801282051282}
```

图 6.6 调参后的模型性能

七、 代码分析和感悟总结

- 1. 模型的输入输出

- **输入：**

  模型的输入是手写数字图片，每张图片的大小是 32x32（代码中通过 resize=32 调整了图片大小）。这些图片会被转换成张量（Tensor）格式，作为神经网络的输入。

- **输出：**

  模型的输出是一个长度为 10 的向量（因为 MNIST 数据集有 10 个类别，对应数字 0 到 9）。这个向量的每个值代表模型认为图片属于某个数字的概率。比如，输出可能是[0.1, 0.8, 0.05, ...]，表示模型认为这张图片是数字 1 的概率最大。

- **2. 模型如何运作**

LeNet-5 模型的结构在代码中通过 LeNet5 类定义，它的工作流程如下：

1. **卷积层（conv1 和 conv2）：**
   - 卷积层的作用是从图片中提取特征。比如，第一层卷积会从原始图片中提取一些简单的特征（如边缘、线条），第二层卷积会提取更复杂的特征（如形状、轮廓）。
   - 代码中，conv1 和 conv2 分别用 5x5 的卷积核扫描图片，提取特征。
2. **激活函数（relu）：**
   - 激活函数的作用是让模型能够学习非线性关系。代码中用的是 ReLU 函数，它会将负值变成 0，保留正值。
3. **池化层（max_pool2d）：**
   - 池化层的作用是降低图片的尺寸，减少计算量。代码中用 2x2 的窗口做最大池化，保留每个窗口中的最大值。
4. **全连接层（fc1, fc2, fc3）：**
   - 全连接层的作用是将提取的特征组合起来，进行分类。前两层全连接（fc1 和 fc2）进一步处理特征，最后一层（fc3）输出 10 个值，对应 10 个数字的概率。
5. **展平操作（flatten）：**
   - 在进入全连接层之前，需要将多维的特征图展平成一维向量，方便

全连接层处理。

- **3. 模型的相关工作**
- **训练：**

  模型通过训练集数据学习如何识别数字。训练过程中，模型会不断调整卷积核和全连接层的参数，使得它的输出（预测概率）尽可能接近真实标签（实际数字）。代码中用交叉熵损失函数（SoftmaxCrossEntropyWithLogits）来衡量预测和真实的差距，并用优化器（Momentum）来更新参数。
- **测试：**

  训练完成后，模型会用测试集数据评估自己的表现。代码中通过 model.eval 计算模型在测试集上的准确率，看看它能不能正确识别别没见过的数字。
- **预测：**

  代码最后用训练好的模型对几张图片进行预测，并画出图片和预测结果。比如，模型可能会预测一张图片是数字“7”，而实际标签也是“7”，说明模型预测正确。

- **4. 结合代码的关键点**
- **输入数据：**

```
images = data["image"].asnumpy()  # 输入图片
labels = data["label"].asnumpy()  # 真实标签
```

这里 images 是模型的输入，labels 是用来对比的真实答案。

- **模型输出：**

```
output = model.predict(ms.Tensor(data['image']))  # 模型输出
predicted = np.argmax(output.asnumpy(), axis=1)  # 取概率最大的类别
```

output 是模型对每张图片的预测概率，predicted 是模型最终预测的数字。

- **模型训练：**

model.train(10, dataset_train, callbacks=[checkpoint, LossMonitor(0.01, 1875)])

这里模型用训练集数据训练 10 轮，过程中会保存模型并监控损失值。

并简要描述下你是通过做了哪些操作让准确率上升的？

通过以下操作可以显著提升模型的准确率：

1. 调整学习率：降低学习率或使用动态学习率，让模型更稳定地收敛。
2. 修改 LeNet 模型架构:增加卷积层、卷积核数量、全连接层、加入 Dropout，提升模型的表达能力。
3. 增加数据量（可选）：通过数据增强或其他数据预处理，让模型学习到更全面的特征。